

实验二：原生应用开发

实验项目基本信息

- **实验编号：**d20301035102、d20301009202
- **学时分配：**4 学时
- **实验类型：**验证型
- **每组人数：**6 人
- **对应课程目标：**课程目标 1

核心步骤列表

实验概览

1. **需求分析阶段：**编写登录功能需求文档（输入验证规则、页面跳转逻辑）
2. **AI 辅助编码阶段：**使用 TRAE 生成登录页面骨架代码并完善业务逻辑
3. **测试验证阶段：**编写输入验证测试用例，验证边界条件
4. **部署运维阶段：**打包调试版本，记录平台差异对比报告

开发任务清单

任务阶段	主要内容	关键技术点
Android 登录	Java + Activity 跳转	EditText、Intent、Bundle
个人界面	学习提醒、智能笔记	SharedPreferences、TextView、EditText
组内对比	技术分享	平台差异分析

☒成功标准

- [] Android 登录页面功能完整
 - [] 个人界面功能完整
 - [] Activity 跳转与数据传递正常
 - [] 输入验证逻辑正确
 - [] 学习提醒和智能笔记功能正常
-

实验任务与案例

核心任务

使用 Java 实现 Android 登录页面（EditText 输入验证、Activity 跳转与数据传递），并在个人界面实现学习提醒和智能笔记功能。

实战案例

开发"在线学习辅助平台"登录模块，用户在登录页面输入用户名和密码，系统验证成功后跳转到个人中心页面，显示学习提醒和智能笔记功能。

第一课时：Android 登录页面设计（2 学时）

2.1 需求分析阶段

步骤 1：编写需求文档

5. 功能需求

- 用户名输入
- 密码输入

- 登录按钮
- 输入验证 (不能为空)

6. 界面需求

- 简洁美观的登录界面
- 输入框占位提示
- 密码隐藏显示

7. 跳转需求

- 登录成功 → 个人中心
- 传递用户名数据

2.2 Android 项目创建

步骤 1: 创建项目

8. 打开 Android Studio
9. 创建 Empty Activity 项目
10. 选择 Java 语言
11. 最低 SDK: API 24

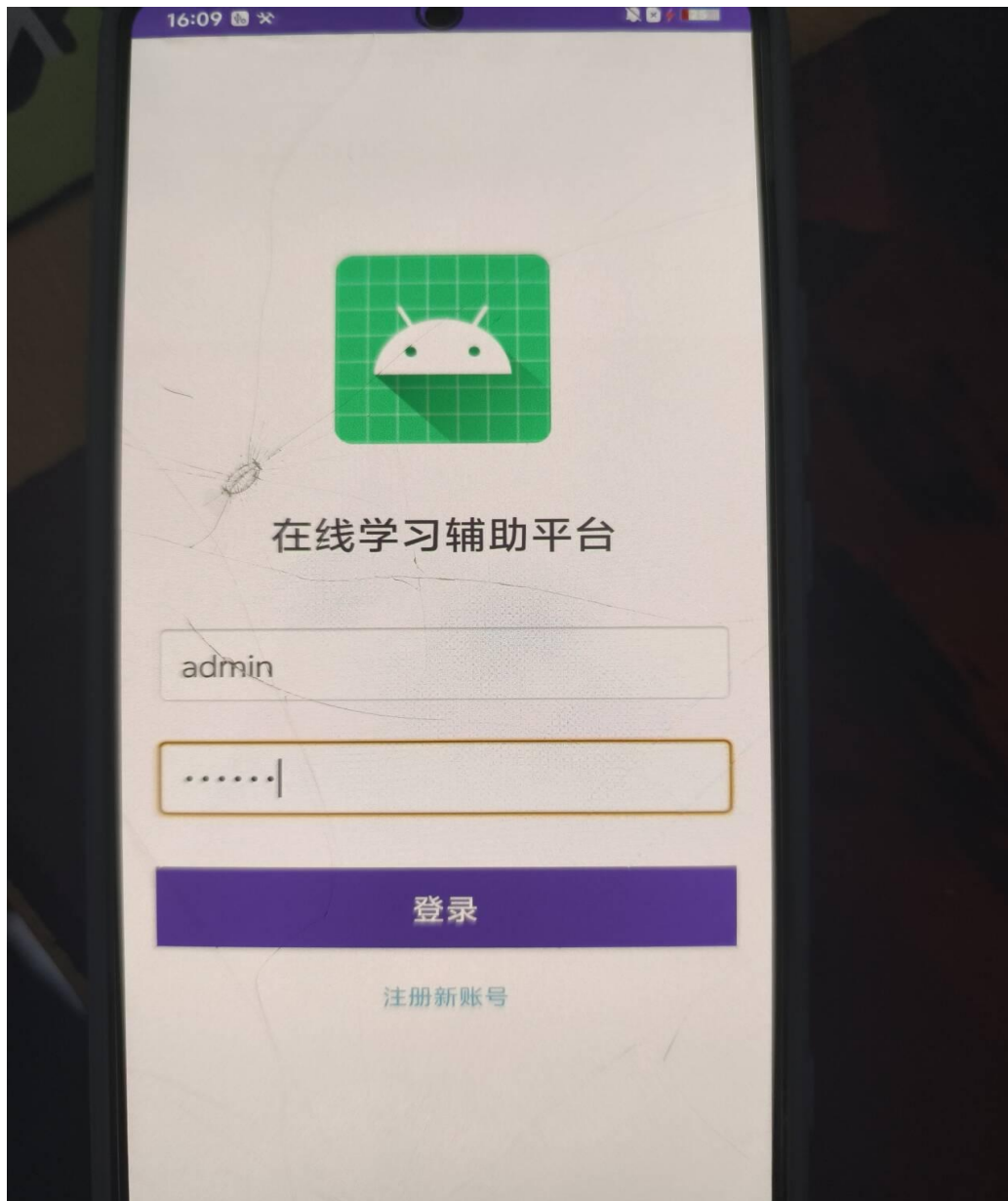
2.3 AI 辅助编码

步骤 1: 生成登录页面代码

12. 使用 TRAE 描述需求: Java 登录页面, EditText 输入验证, Activity 跳转
13. AI 生成代码片段
14. 手动完善业务逻辑

步骤 2：实现布局

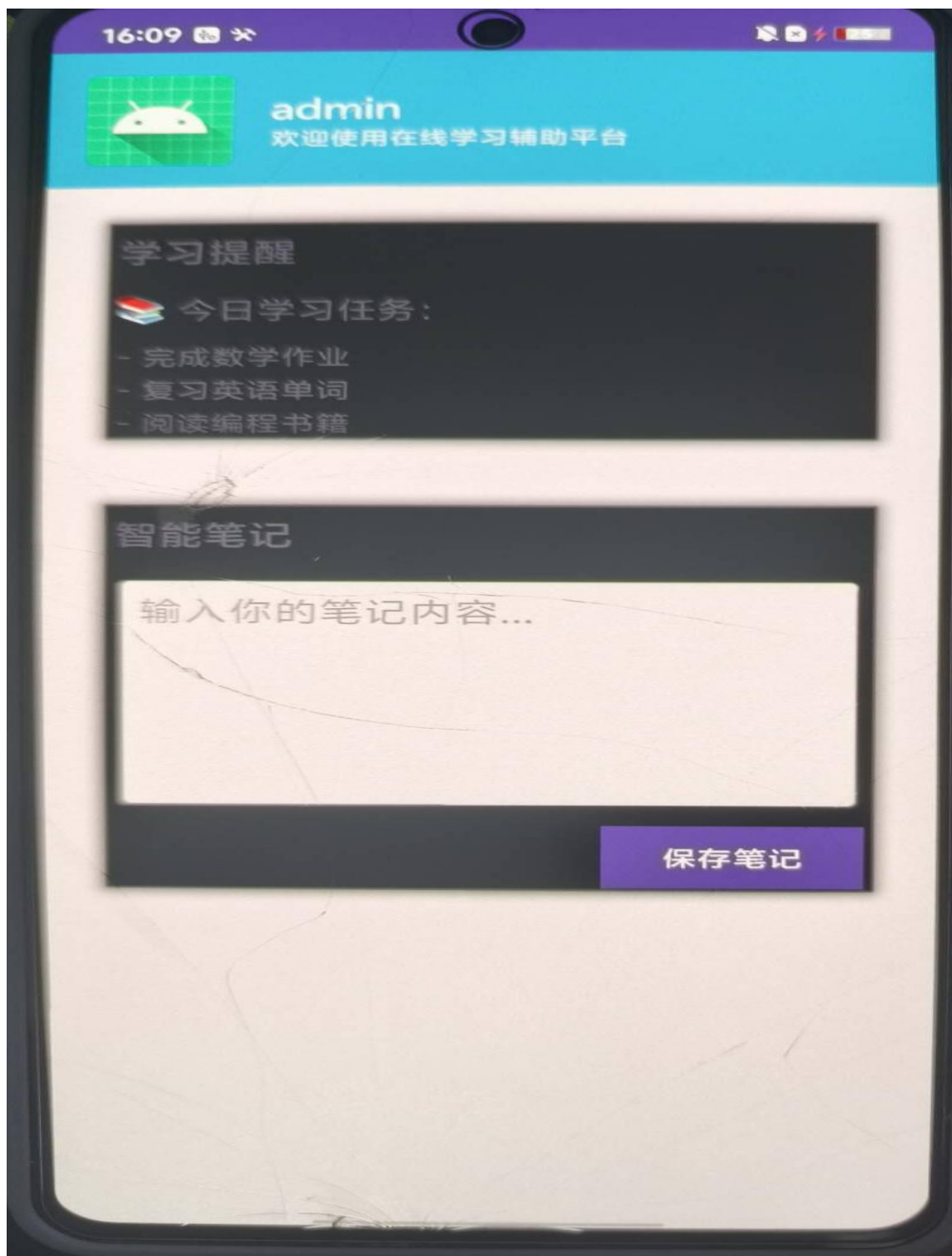
登录界面截图



2.4 创建个人中心页面

步骤 1：创建 PersonalActivity

15. 新建 Activity：PersonalActivity
16. 布局显示欢迎信息、学习提醒和智能笔记



个人中心界面截图

第二课时：测试验证与平台对比（2 学时）

2.5 测试验证阶段

步骤 1：编写测试用例

- 17. 输入测试
- 18. 为空测试
- 19. 正确凭证测试
- 20. 错误凭证测试

步骤 2：执行测试

- 21. 在模拟器上运行
- 22. 验证边界条件
- 23. 记录测试结果

2.6 平台差异对比

步骤 1：整理对比报告

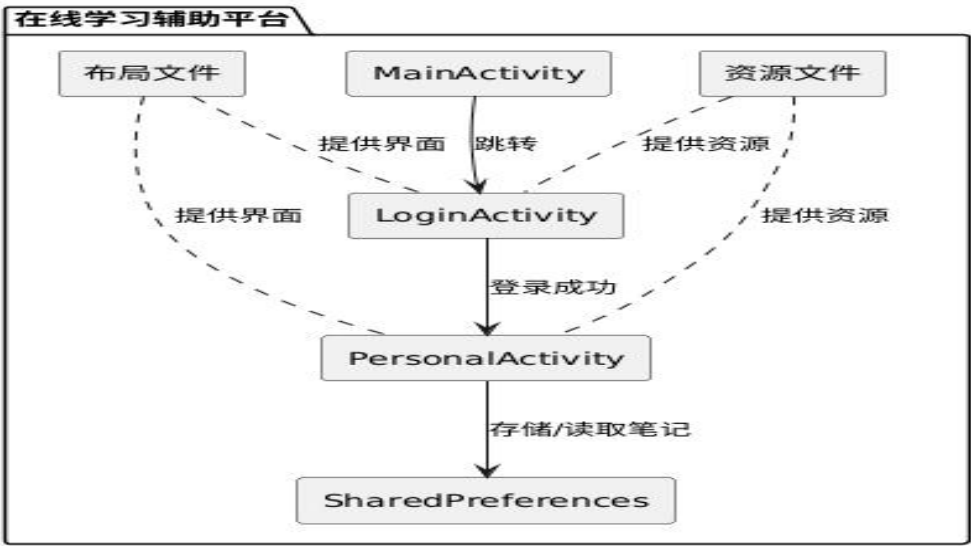
对比项	Android	iOS
开发语言	Java	Swift
UI 框架	XML	SwiftUI
页面跳转	Intent	NavigationLink
数据传递	Intent.putExtra	@Binding
数据存储	SharedPreferences	UserDefaults

步骤 2：组内技术分享

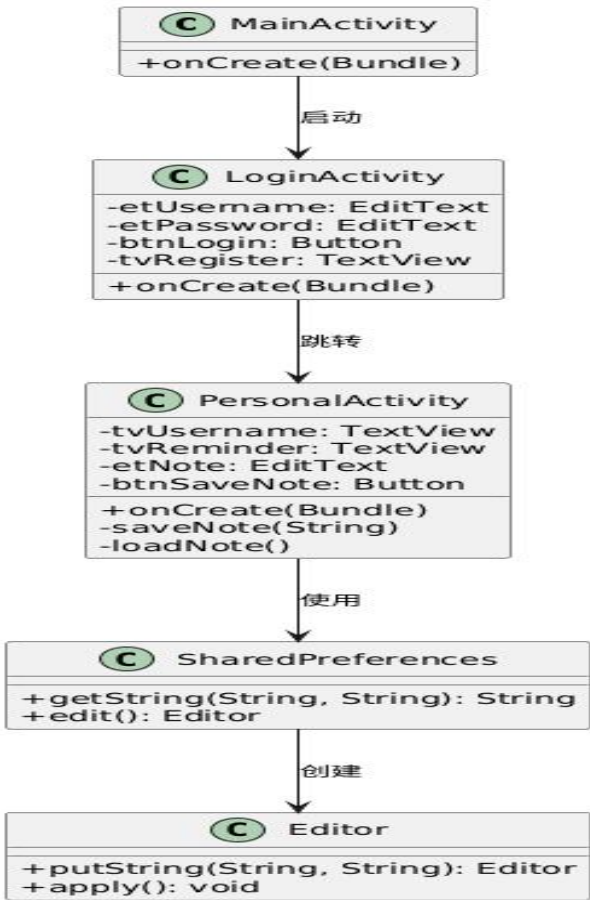
- 24. 各成员介绍实现方案
 - 25. 讨论平台差异
 - 26. 总结技术选型考虑
-

系统设计

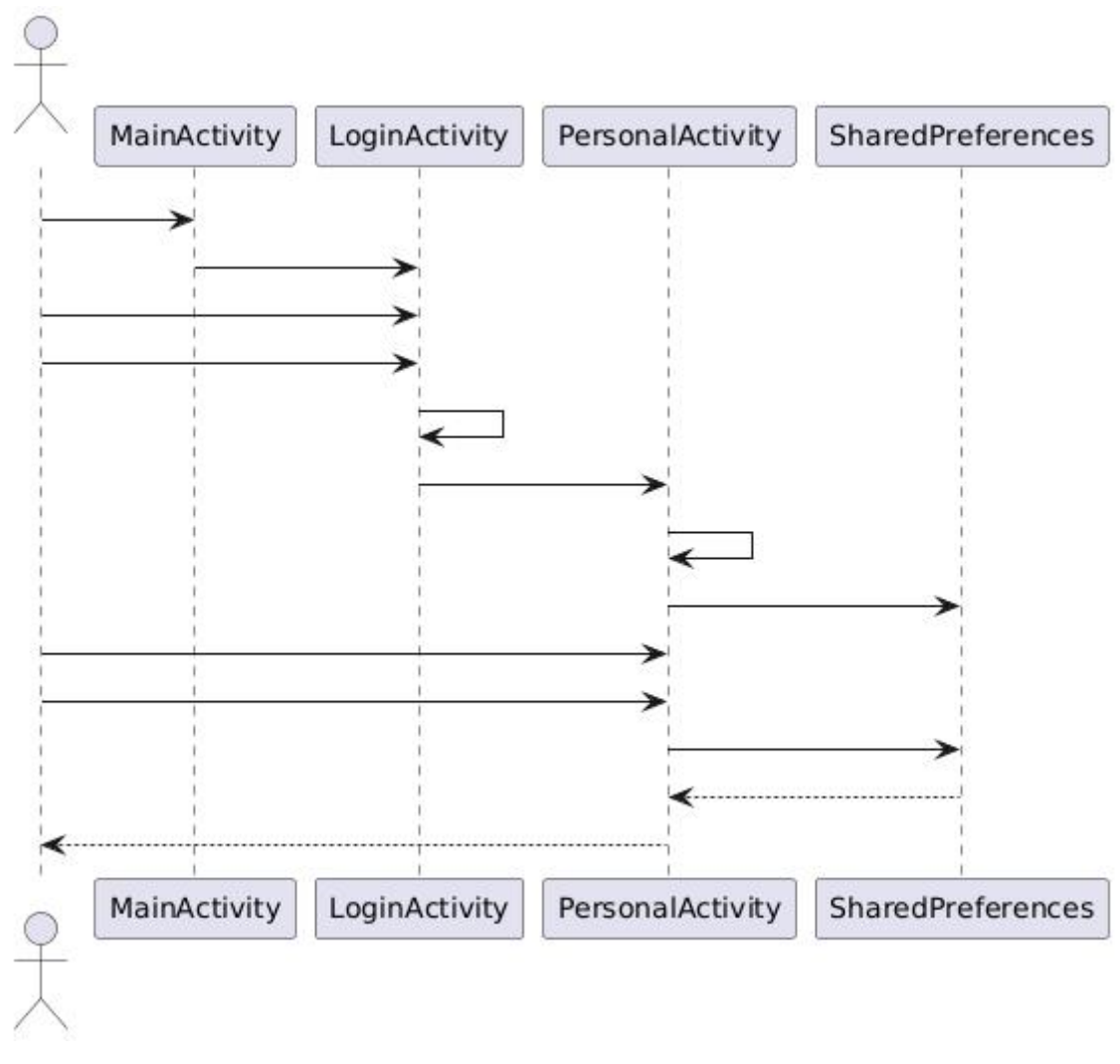
框架图



类图



顺序图



总结与思考

实验总结

- 掌握 Android Java 登录页面开发
- 理解 Activity 间数据传递
- 实现学习提醒和智能笔记功能
- 学会使用 SharedPreferences 进行数据存储

课后思考

27. 原生开发相比跨平台的优势和劣势
28. 如何选择原生开发 vs 跨平台开发
29. AI 辅助代码生成对开发效率的影响
30. 如何进一步优化学习提醒和智能笔记功能